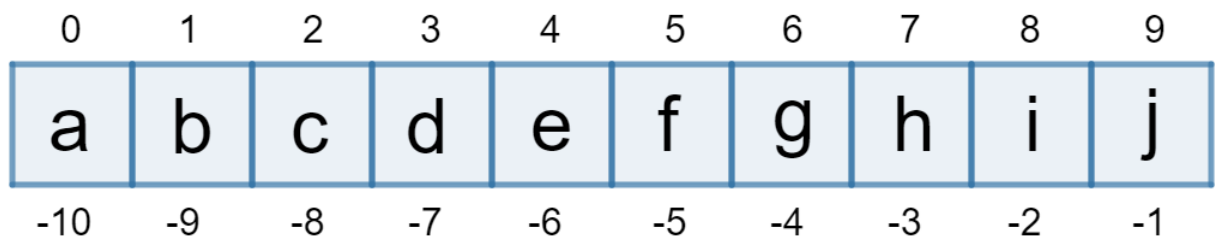


Срезы строк

В предыдущем уроке мы научились работать с конкретными символами строки с помощью индексов `[]`. Иногда нужно бывает работать с целыми частями строки, в таком случае мы используем **срезы (slices)**. Срезы похожи на комбинацию индексации и функции `range()`.

Рассмотрим строку `s = 'abcdefghij'`.

Положительные индексы	0	1	2	3	4	5	6	7	8	9
Строка	a	b	c	d	e	f	g	h	i	j
Отрицательные индексы	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1



С помощью среза мы можем получить несколько символов исходной строки, создав диапазон индексов разделенных двоеточием `s[x:y]`.

Следующий программный код:

```
print(s[2:5])
```

```
print(s[0:6])
```

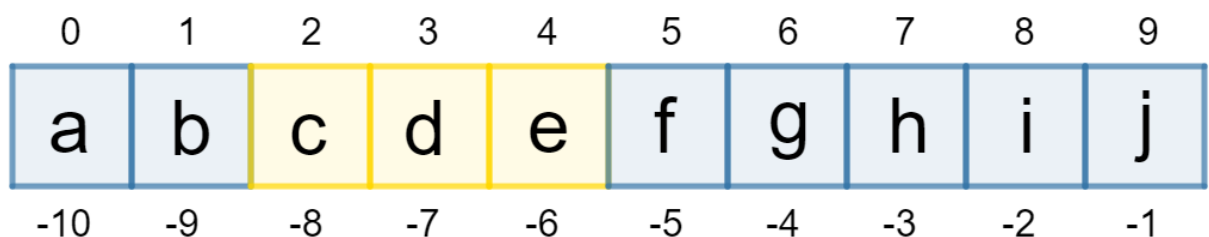
```
print(s[2:7])
```

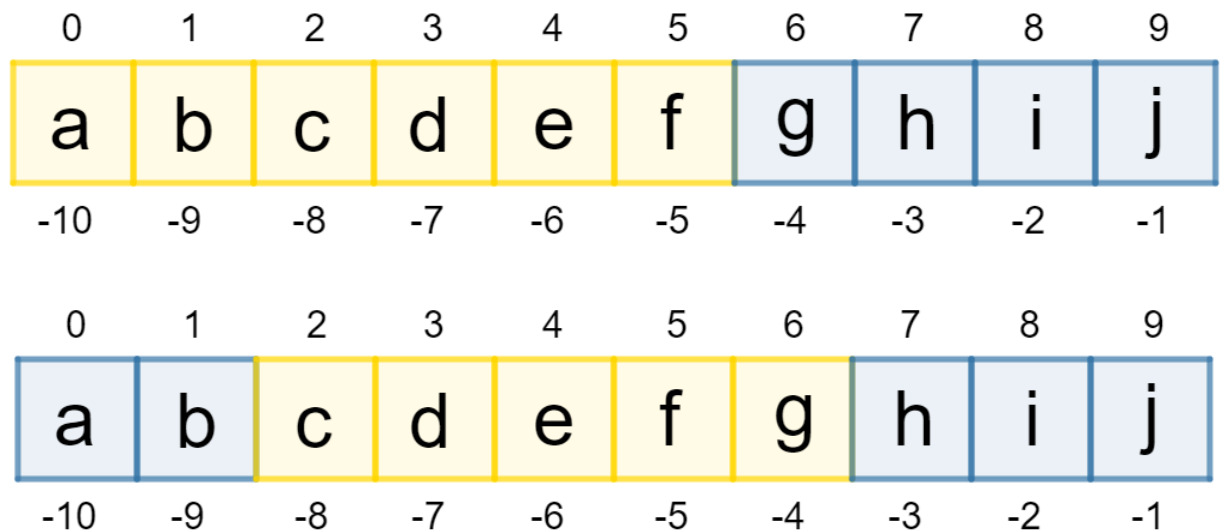
ВЫВОДИТ:

```
cde
```

```
abcdef
```

```
cdefg
```





При построении среза `s[x:y]` первое число – это то место, где начинается срез (**включительно**), а второе – это место, где заканчивается срез (**невключительно**). Разрезая строки, мы создаем подстроку, которая по сути является строкой внутри другой строки.

Срез до конца, от начала

Если опустить второй параметр в срезе `s[x:]` (но поставить двоеточие), то срез берется до конца строки. Аналогично если опустить первый параметр `s[:y]`, то можно взять срез от начала строки. Срез `s[:]` совпадает с самой строкой `s`.

Следующий программный код:

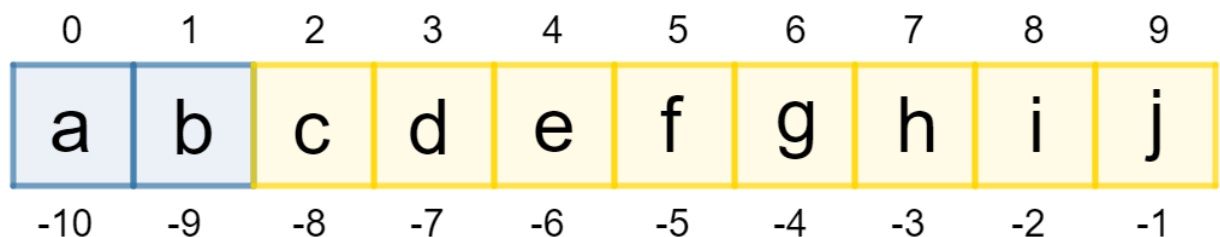
```
print(s[2:])
```

```
print(s[:7])
```

ВЫВОДИТ:

```
cdefghij
```

```
abcdefg
```



0	1	2	3	4	5	6	7	8	9
a	b	c	d	e	f	g	h	i	j
-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Срез `s[:]` возвращает исходную строку.

Отрицательные индексы в срезе

Мы также можем использовать отрицательные индексы для создания срезов. Как уже говорилось ранее, отрицательные индексы строки начинаются с `-1` и отсчитываются до достижения начала строки. При использовании отрицательных индексов **первый параметр среза должен быть меньше второго, либо должен быть пропущен.**

Следующий программный код:

```
print(s[-9:-4])
```

```
print(s[-3:])
```

```
print(s[:-3])
```

ВЫВОДИТ:

```
bcdef
```

```
hij
```

```
abcdefg
```

0	1	2	3	4	5	6	7	8	9
a	b	c	d	e	f	g	h	i	j
-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

0	1	2	3	4	5	6	7	8	9
a	b	c	d	e	f	g	h	i	j
-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

0	1	2	3	4	5	6	7	8	9
a	b	c	d	e	f	g	h	i	j
-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Удалить из строки последний символ можно при помощи среза `s[:-1]`.

Шаг среза

Мы можем передать в срез третий необязательный параметр, который отвечает за шаг среза. К примеру, срез `s[1:7:2]` создаст строку `bdf` состоящую из каждого второго символа (индексы `1, 3, 5`, правая граница не включена в срез).

0	1	2	3	4	5	6	7	8	9
a	b	c	d	e	f	g	h	i	j
-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Отрицательный шаг среза

Если в качестве шага среза указать **отрицательное число**, то символы будут идти в обратном порядке.

Следующий программный код:

```
print(s[::-1])
```

ВЫВОДИТ:

```
jihgfedcba
```

Следующий программный код:

```
print(s[1:7:2])
```

```
print(s[3::2])
```

```
print(s[:7:3])
```

```
print(s[::2])
```

```
print(s[::-1])
```

```
print(s[::-2])
```

ВЫВОДИТ:

```
bdf
dfhj
adg
acegi
jihgfedcba
jhfdb
```

Подводя итог

```
s = 'abcdefghij'
```

Программный код	Результат	Пояснение
<code>s[2:5]</code>	<code>cde</code>	строка состоящая из символов с индексами 2, 3, 4
<code>s[:5]</code>	<code>abcde</code>	первые пять символов строки
<code>s[5:]</code>	<code>fghij</code>	строка состоящая из символов с индексами от 5 до конца
<code>s[-2:]</code>	<code>ij</code>	последние два символа строки
<code>s[:]</code>	<code>abcdefghij</code>	вся строка целиком
<code>s[1:7:2]</code>	<code>bdf</code>	строка состоящая из каждого второго символа с индексами от 1 до 6
<code>s[::-1]</code>	<code>jihgfedcba</code>	строка в обратном порядке, так как шаг отрицательный

Изменение символа строки по индексу

Предположим, у нас есть строка `s = 'abcdefghij'` и мы хотим заменить символ с индексом 4 на `'X'`. Можно попытаться написать код:

```
s[4] = 'X'
```

Однако такой код не работает. В Python строки являются **неизменяемыми**, то есть мы не можем менять их содержимое с помощью индексатора.

Если мы хотим поменять какой-либо символ строки `s`, мы должны создать новую строку. Следующий код использует срезы и решает поставленную задачу:

```
s = s[:4] + 'X' + s[5:]
```

Мы создаем два среза: от начала строки до 3 символа и с 5 символа по конец строки, а между ними вставляем нужный нам символ, который встанет на 4 позицию.

Примечания

Примечание 1. Синтаксис срезов строк очень похож на синтаксис функции `range()`.

Примечание 2. Если первый параметр среза больше второго, то срез создает пустую строку.